3. Design

냉장GO

Student No	22311970
Name	김나은
E-mail	migon0329@naver.com

[Revision history]

Revision date	Version #	Description	Author
03/26/2025	1.00	First draft	김나은
05/01/2025	1.01	일부 기능 수정	김나은
05/14/2025	1.02	class diagram, state machine diagram, sequence diagram 수정	김나은

= Contents =

1. Introduction .....

2. Class diagram .....

3. Sequence diagram .....

4. State machine diagram .....

5. Implementation requirements .....

6. Glossary .....

7. References .....

1. Introduction

1) Summary

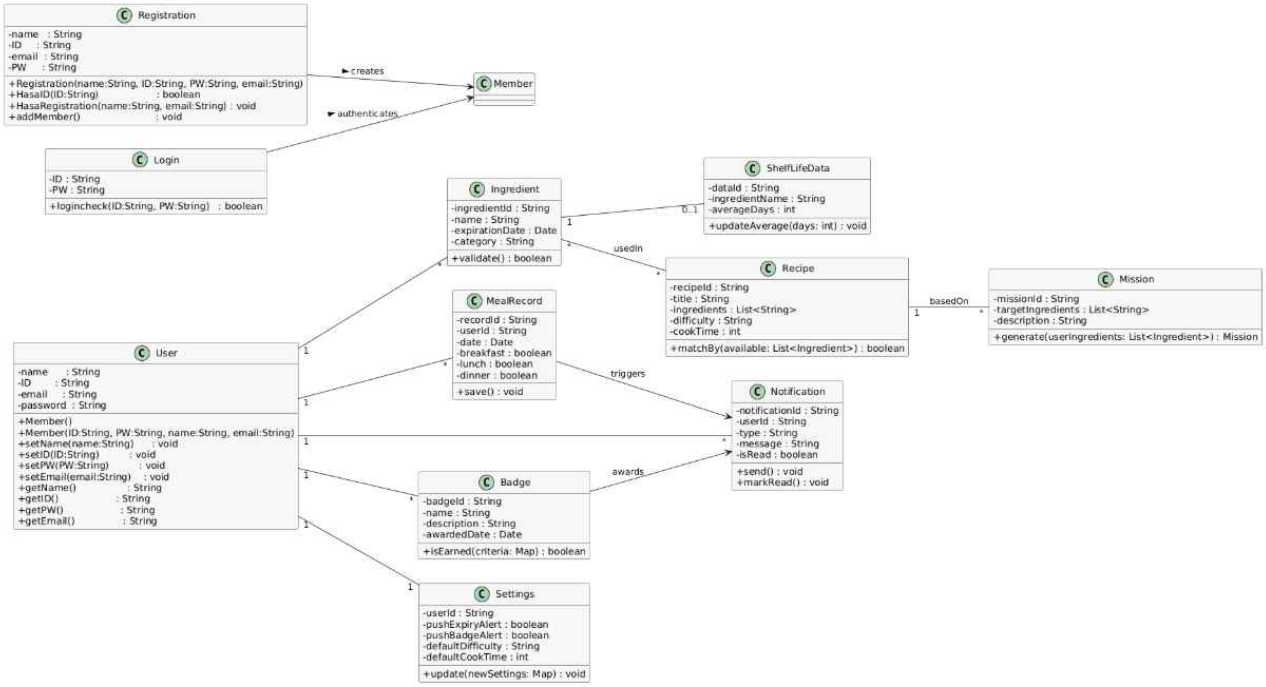
1인 가구, 특히 자취생에게는 식재료를 체계적으로 소비하기가 쉽지 않다. 냉장고에 방치된 재료가 언젠가 상하거나 유통기한이 지나 버리는 일이 반복되며, 이는 경제적 손실과 식품 낭비로 이어진다. 유통기한 임박 알림 기능이 이미 존재하지만, 단순히 날짜만 알려주는 수준을 넘어서는 웹 서비스를 목표로 한다. 이 서비스는 기한이 다가온 재료에 대한 요리 제안, 생활 패턴에 기반한 적극적 리마인드, 그리고 집밥 실천 일수를 기록 및 시각화해 주는 피드백을 제공하여 사용자가 자신의 소비 습관을 돌아보도록 돕는다. 주요 대상은 자취생이나 1인 가구, 대학생 등 식재료를 직접 구매 및 관리하는 이들이지만, 궁극적으로 음식물 쓰레기를 줄이고 집밥 문화를 확산하고자 하는 누구에게나 유용할 것으로 기대된다.

이 문서는 Analysis 단계에 이어 이 시스템 개발의 세 번째 단계인 Design에 대한 내용이다. Design 단계에서는 실제 시스템 구현에 필요한 모든 구성 요소의 개요를 확정하고, 이를 Diagram을 통해 구체적인 설계로 발전시키는 과정을 다룬다.

2) Important Points of Design

- 사용자 경험(UX)를 최적화하기 위해 간단하고 직관적인 네비게이션이 필요하다.
- 버튼, 입력폼, 알림창 등 일관된 UI 컴포넌트로 학습 곡선을 완화해야 한다.

2. Class diagram



User	**Attributes** 1) ID : String – 사용자 고유 식별자 2) name : String – 이름 3) email : String – 로그인용 이메일 4) password : String – 해시된 비밀번호
	Methods 1) User() : 생성자 - 기본 생성자 2) User(ID, PW, name, email) : 생성자 - 네 개 필드를 초기화하는 생성자 3) setName(Name) : void - name 설정 4) setID(ID) : void - ID 설정 5) setPW(PW) : void - password 설정 6) setEmail(email) : void - email 설정 7) getName() : String - name 반환 8) getID() : String - ID 반환 9) getPW() : String - password 반환 10) getEmail() : String - email 반환
Registration	**Attributes** 1) name : String - 가입 요청 시 입력된 사용자 이름 2) ID : String - 가입 요청 시 입력된 사용자 ID

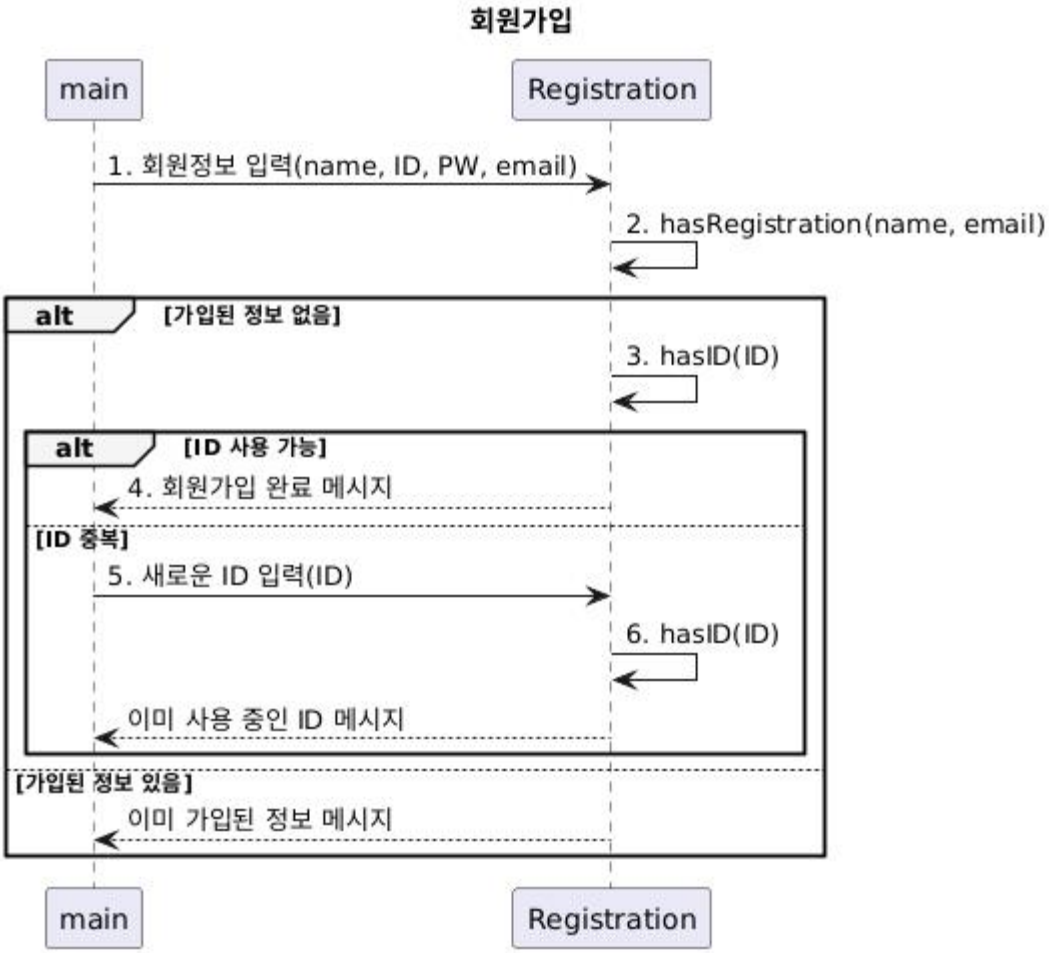
	3) email : String - 가입 요청 시 입력된 이메일 4) PW : String - 가입 요청 시 입력된 비밀번호
	Methods 1) Registration(...) : void Registration(String name, String ID, String PW, String email) - 네 개 필드를 받아 회원가입 프로세스 시작 2) HasaID(ID) : boolean HasaID(String ID) - 주어진 ID 중복 여부 검사 3)HasaRegistration(name,email) :void HasaRegistration(String name, String email) - name+email 조합으로 이미 가입된 정보인지 확인 4) addMember() : void addMember() - 내부 저장소에 새로운 User 객체를 추가
Login	**Attributes** 1) ID : String - 로그인 시 입력받는 사용자 ID 2) PW : String - 로그인 시 입력받는 비밀번호
	Methods 1) logincheck(ID, PW) : boolean, logincheck(String ID, String PW) - 입력된 ID/PW의 일치 여부를 확인하고 결과 반환
Ingredient	**Attributes** 1) ingredientId : String — 식재료 고유 식별자 2) name : String — 식재료명 3) expirationDate : Date — 유통기한 4) category : String — 분류 (채소·과일 등)
	Methods 1) validate() : boolean — 입력된 유통기한 및 필수 속성 유효성 검사
ShelfLifeData	**Attributes** 1) dataId : String — 데이터 고유 식별자 2) ingredientName : String — 해당 식재료명 3) averageDays : int — 평균 보존일수
	Methods 1) updateAverage(days:int) : void — 새로운 보존일수 반영하여 평균 갱신
Recipe	**Attributes** 1) recipeId : String — 레시피 고유 식별자

	2) title : String – 요리 제목 3) ingredients : List<String> – 필요 재료명 목록 4) difficulty : String – 난이도 (ex: 쉬움/보통/어려움) 5) cookTime : int – 예상 조리 시간
	Methods 1) matchBy(available:List<Ingredient>) : boolean – 사용자의 보유 재료 목록과 비교하여 조리 가능 여부 판단
Mission	**Attributes** 1) missionId : String – 미션 고유 식별자 2) targetIngredients : List<String> – 소비 대상 재료명 목록 3) description : String – 미션 설명
	Methods 1) generate(userIngredients:List<Ingredient>) : Mission – 사용자 보유 재료를 바탕으로 임박 재료 미션 생성
Settings	**Attributes** 1) userId : String – 소유 사용자 ID 2) pushExpiryAlert : boolean – 유통기한 알림 여부 3) pushBadgeAlert : boolean – 뱃지 알림 여부 4) defaultDifficulty : String – 기본 레시피 난이도 설정 5) defaultCookTime : int – 기본 조리 시간 설정
	Methods 1) update(newSettings:Map) : void – 변경된 설정값을 저장
MealRecord	**Attributes** 1) recordId : String – 기록 고유 식별자 2) userId : String – 기록한 사용자 ID 3) date : Date – 기록 날짜 4) breakfast : boolean – 아침 집밥 여부 5) lunch : boolean – 점심 집밥 여부 6) dinner : boolean – 저녁 집밥 여부
	Methods 1) save() : void – 기록 저장
Notification	**Attributes** 1) notificationId : String – 알림 고유 식별자 2) userId : String – 대상 사용자 ID

	3) type : String – 알림 유형(“expiry”, “badge” 등) 4) message : String – 알림 메시지 5) isRead : boolean – 읽음 상태
	Methods 1) send() : void – 알림 전송 2) markRead() : void – 읽음 처리
Badge	**Attributes** 1) badgeId : String – 뱃지 고유 식별자 2) name : String – 뱃지 이름 3) description : String – 뱃지 설명 4) awardedDate : Date – 획득 일시
	Methods 1) isEarned(criteria:Map) : boolean – 주어진 조건(연속 달성일 등)에 따른 획득 여부 판단

3. Sequence diagram

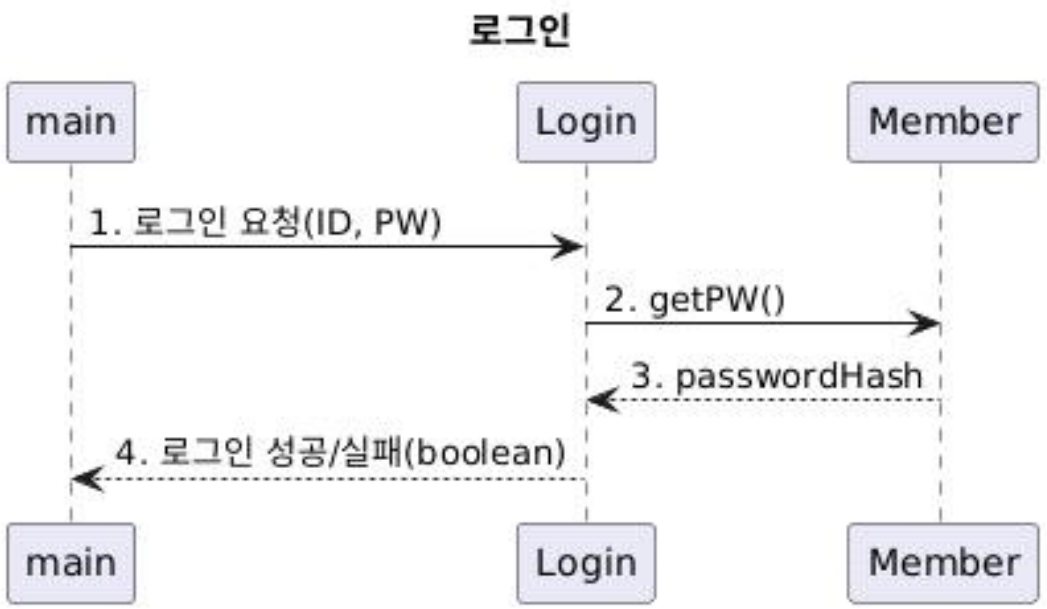
1) 회원가입



위의 그림은 시스템 실행 후 회원가입 기능을 수행할 때를 표현한 Sequence Diagram이다.

1. **회원정보 입력** : main이 Registration에 사용자명, ID, PW, 이메일을 전달한다.
2. **가입 정보 중복 검사** : Registration이 hasRegistration(name, email)으로 이름+이메일 조합이 이미 등록되었는지 확인한다.
3. **가입 정보가 없으면 ID 중복 검사** : Registration이 hasID(ID)로 입력된 ID의 사용 가능 여부를 확인한다.
4. **ID 사용 가능 시** : Registration이 main에 "회원가입 완료" 메시지를 반환한다.
5. **ID 중복 시** : main이 Registration에 새로운 ID를 입력하고, 다시 hasID(ID)를 호출하여 중복을 재확인한다. 여전히 중복이면 "이미 가입된 ID" 메시지를 main에 전달한다.
6. **이름+이메일 조합이 이미 등록된 경우** : 바로 "이미 가입된 정보" 메시지를 main에 반환한다.

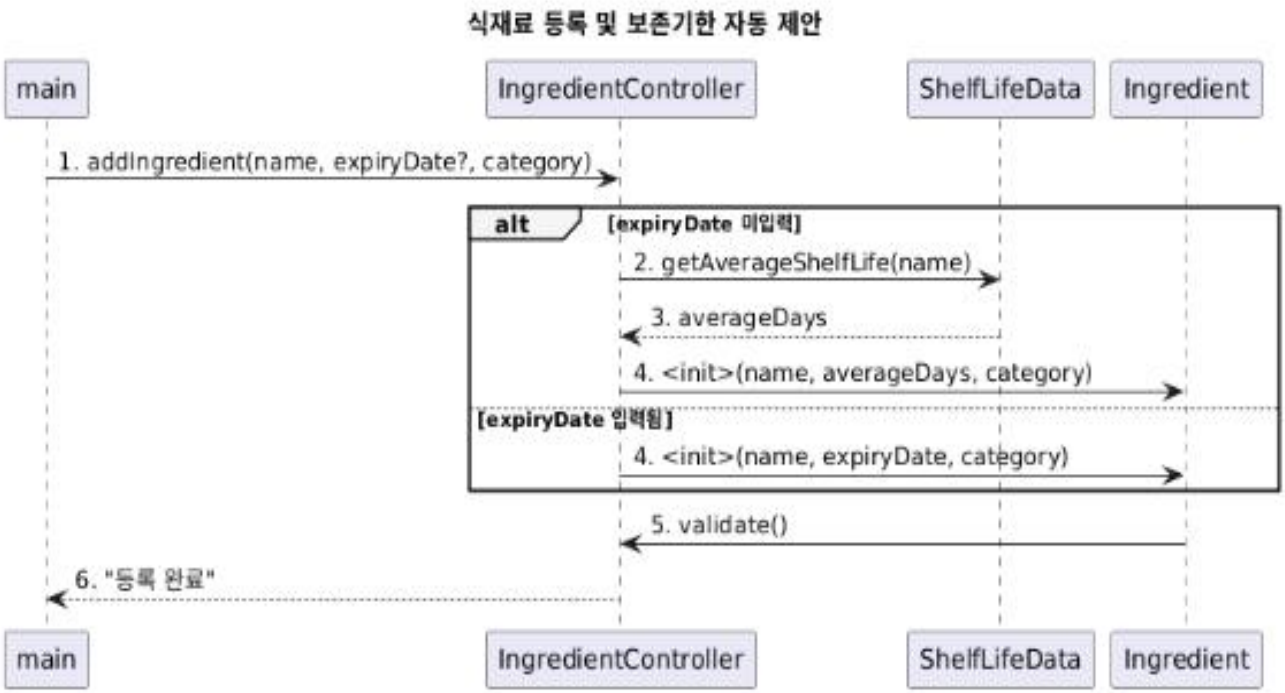
2) 로그인



위의 그림은 회원가입을 완료한 후 시스템 실행 중에 로그인 기능을 수행할 때를 표현한 Sequence Diagram이다.

1. main에서 Login에 ID/PW를 보낸다.
2. Login이 저장된 해시된 비밀번호를 조회한다.
3. PW 일치 여부를 판단해 성공이면 true, 실패면 false를 반환한다.

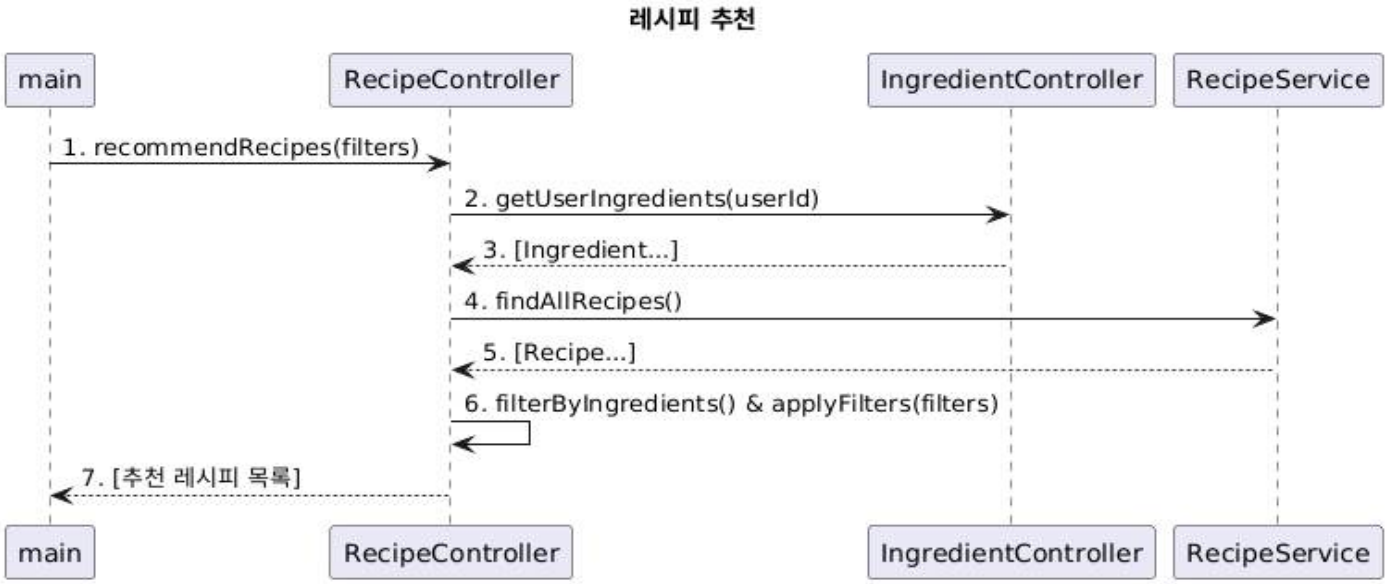
3) 식재료 등록 및 보존기한 자동 제안



위의 그림은 사용자가 로그인 후 식재료 등록 기능을 수행할 때 및 식재료를 등록할 때 유통기한이 비어있는 경우 보존기한을 자동으로 제안하는 기능을 수행할 때를 표현한 Sequence Diagram이다.

1. 유통기한(expiryDate) 미입력 시 ShelfLifeData에서 평균값을 가져와 Ingredient 객체를 생성한다.
2. 생성된 Ingredient를 검증(validate())한 뒤 저장 및 완료 메시지를 반환한다.

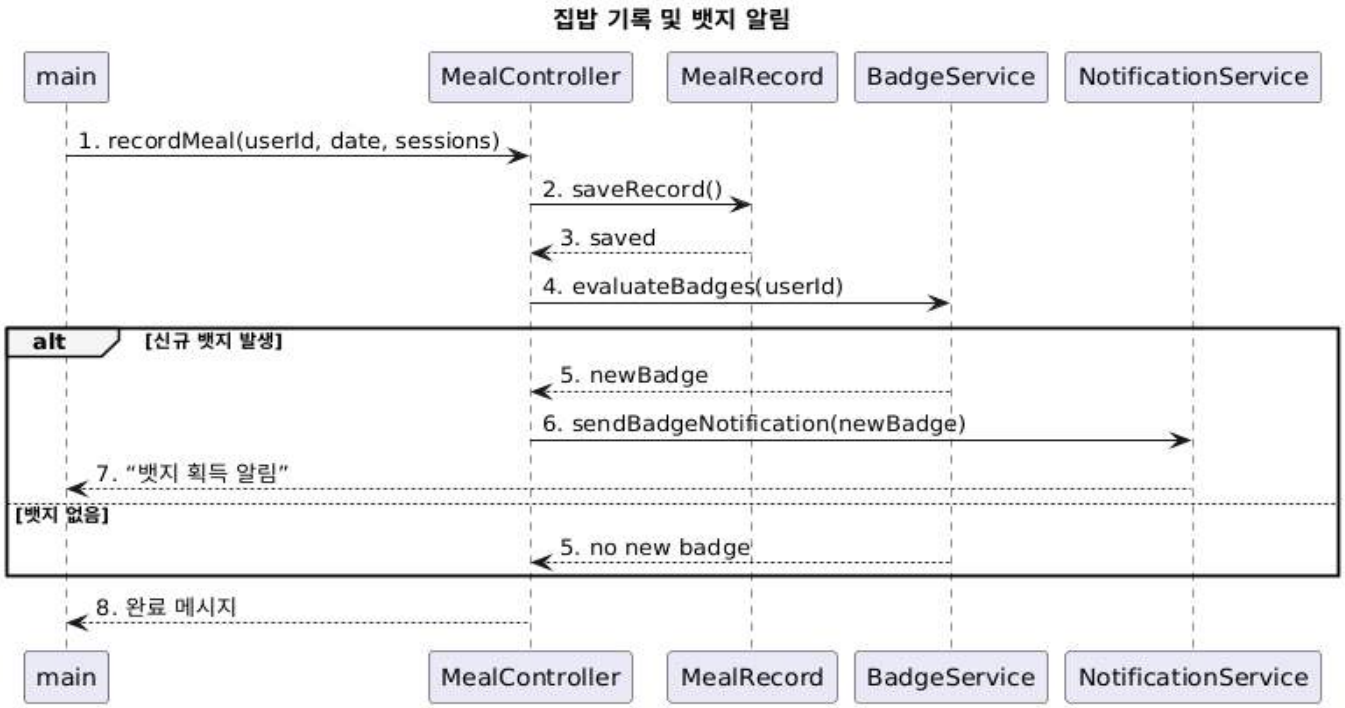
4) 레시피 추천



위의 그림은 최소 한 개 이상의 식재료가 등록되어 있을 경우 레시피 추천 기능을 수행할 때를 표현한 Sequence Diagram이다.

1. 보유 식재료 목록을 조회한다.
2. 모든 레시피를 가져온다.(findAllRecipes())
3. 보유 재료 및 필터 조건으로 걸러낸 뒤 결과를 반환한다.

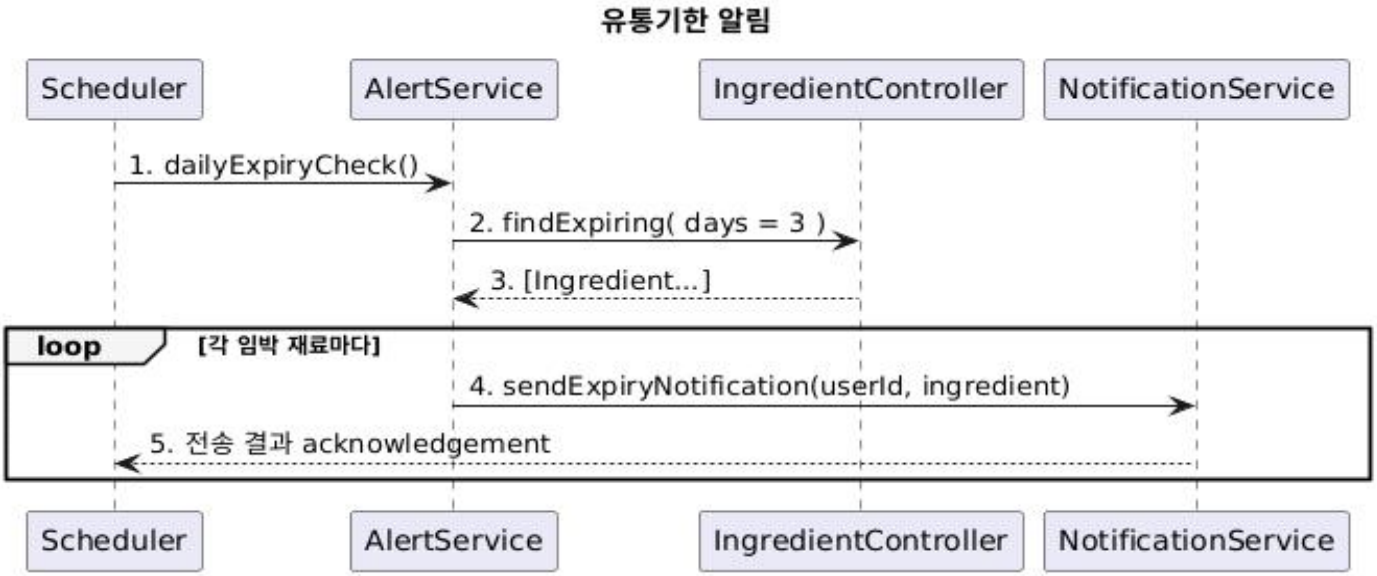
5) 집밥 기록 및 뱃지 알림



위의 그림은 사용자의 집밥 데이터가 있을 경우 집밥 기록 및 뱃지 알림 기능을 수행할 때를 표현한 Sequence Diagram이다.

1. 집밥 기록을 저장한다.
2. BadgeService로 뱃지 조건을 평가한다.
3. 신규 뱃지 발생 시 NotificationService를 통해 사용자에게 알림을 보낸다.

6) 유통기한 알림



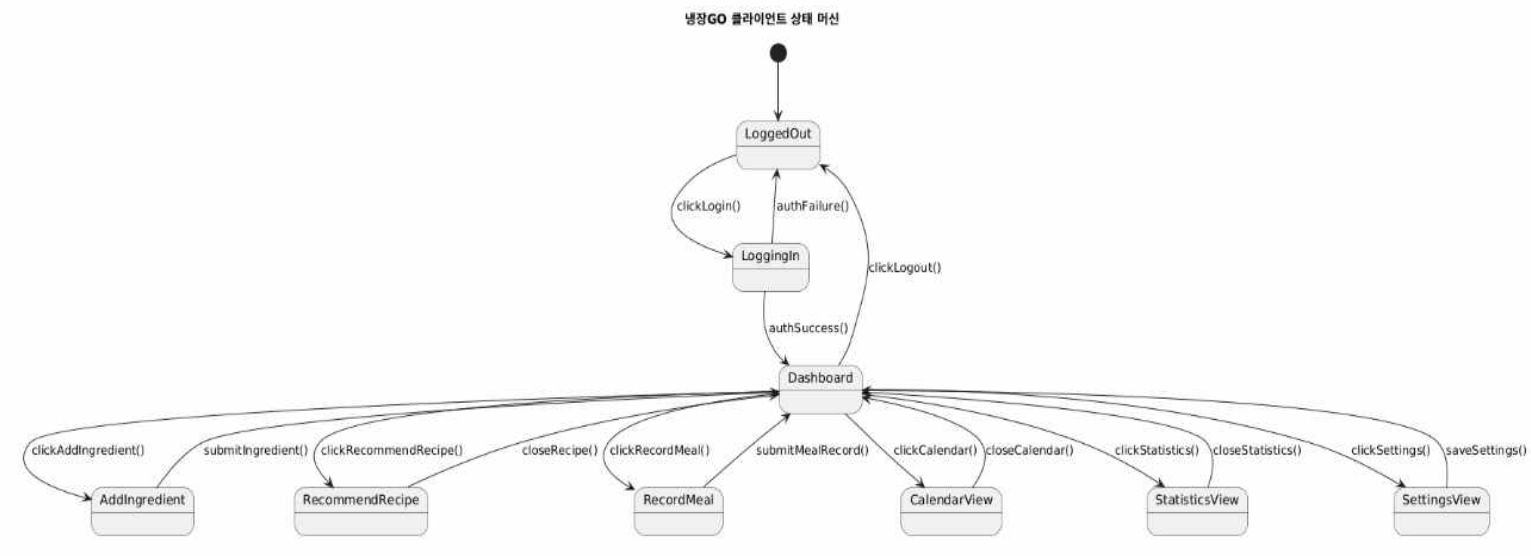
위의 그림은 등록된 식재료가 최소 한 개 이상일 경우 유통기한 알림 기능을 수행할 때를 표현한 Sequence Diagram이다.

1. 스케줄러가 매일 dailyExpiryCheck()를 호출한다.
2. 임박 식재료를 조회한다.
3. 각 사용자에게 푸시/인앱 알림을 전송한다.

4. State machine diagram

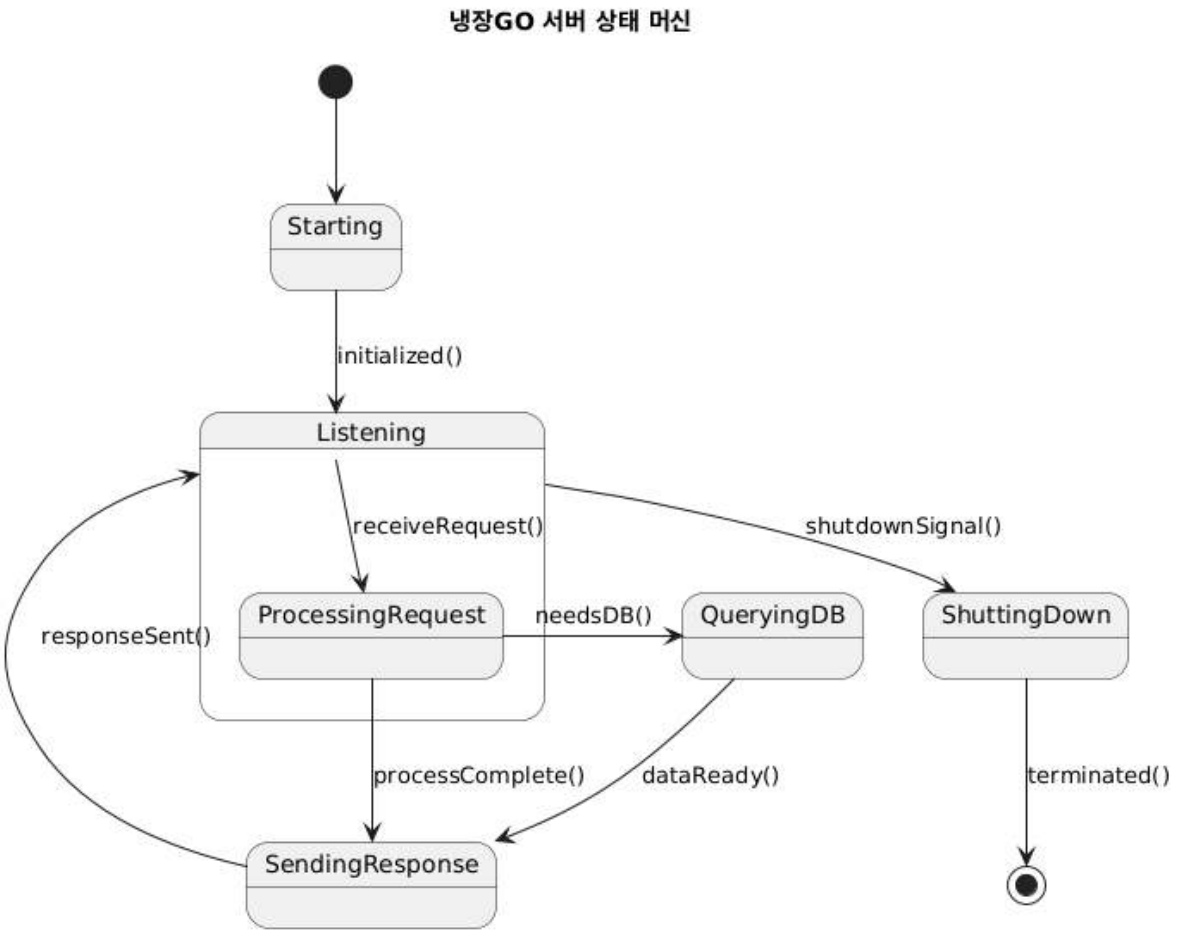
아래의 그림은 각각 냉장GO 시스템의 State machine diagram을 클라이언트와 서버 시스템으로 구분하여 표현한 그림이다.

1) 클라이언트 상태 머신 다이어그램



냉장GO 클라이언트는 처음 LoggedOut 상태에서 시작하여, 사용자가 로그인 버튼을 누르면 LoggingIn 상태로 전이한다. 이때 서버 인증 결과가 실패하면 다시 LoggedOut으로 돌아가고, 성공하면 Dashboard(메인 화면)로 진입한다. 대시보드에서는 “식재료 등록”, “레시피 추천”, “집밥 기록”, “캘린더 보기”, “통계 보기”, “설정” 등 각 기능 버튼을 클릭할 때마다 해당 기능의 상태(AddIngredient, RecommendRecipe, RecordMeal, CalendarView, StatisticsView, SettingsView)로 전환했다가, 작업 완료(등록,저장,닫기 등) 이벤트가 발생하면 다시 Dashboard로 복귀한다. 마지막으로, 사용자가 로그아웃을 선택하면 LoggedOut 상태로 돌아가 화면이 로그인 화면으로 전환된다.

2) 서버 상태 머신 다이어그램



냉장GO 서버는 부팅 직후 Starting 상태에서 초기화 작업을 마치면 Listening (요청 대기) 상태로 들어간다. 클라이언트로부터 REST API 요청이 들어오면 ProcessingRequest 상태로 이동하여 비즈니스 로직을 수행한다. 이 과정에서 데이터베이스 접근이 필요하면 QueryingDB 상태로 잠시 전이했다가, DB 조회/저장 작업이 끝나면 SendingResponse 상태로 넘어가 응답을 조립·전송한다. 응답이 완료되면 다시 Listening 상태로 복귀하여 다음 요청을 기다리고, 운영자가 종료 신호를 보내면 ShuttingDown 상태로 전환되어 모든 리소스를 정리한 뒤 정상 종료(terminated())로 마무리한다.

5. Implementation requirements

1) Hardware Requirements

CPU	2코어 이상
RAM	4GB 이상
Storage	20GB 이상의 여유 디스크 공간
Network	인터넷 연결 (웹 푸시 알림/외부 API 호출용)

2) Software Requirements

Operating System	Windows 10 이상
Implementation Language	Java

3) Nonfunctional Requirements

본 시스템은 별도의 백엔드 서버 없이, Sever 역할 모듈이 모든 비즈니스 로직을 수행하며, 클라이언트 자체가 요청을 처리한다. 입력 방식 측면에서, 사용자가 수동으로 신규 식재료나 설정을 입력할 경우, 입력값은 LocalStorage 또는 IndexedDB에 저장된다. 수동 입력 시 날짜 형식(YYYY-MM-DD)을 엄격히 준수해야 하며, 그렇지 않으면 입력이 무시되거나 오류 팝업이 발생한다. 또, 오류 처리의 측면에서, 파일 읽기 실패, 형식 불일치, LocalStorage 접근 오류 등 모든 예외 상황에 대해 경고창(alert) 또는 인앱 토스트 메시지를 통해 사용자에게 즉시 알린다.

6. Glossary

Attribute	클래스 내부에 정의된 변수로, 객체의 상태를 나타내는 데이터 요소이다.
Class Diagram	UML에서 클래스와 그 관계(연관, 상속, 집합 등)를 시각적으로 표현한 다이어그램으로, 시스템의 정적 구조를 설계 및 문서화할 때 사용한다.
Sequence Diagram	UML에서 객체(또는 컴포넌트) 간에 교환되는 메시지의 흐름과 순서를 시간축에 따라 나타낸 다이어그램으로, 동적 행위(시나리오)를 모델링한다.
Sate Machine Diagram	UML에서 객체나 시스템이 가질 수 있는 상태들과 상태 전이를 표현한 다이어그램으로, 이벤트 발생에 따른 상태 변화를 설계 및 검증할 때 사용한다.
Method	클래스 내부에 정의된 함수로, 객체가 수행할 수 있는 동작을 구현한 코드 블록이다.

7. References

file:///C:/Users/PC/AppData/Local/Temp/BNZ.682fbeb5d9b17/3.%20[Design]%20Example%201.pdf
file:///C:/Users/PC/AppData/Local/Temp/BNZ.682fbeb6d9fcb/3.%20[Design]%20Example%202.pdf
file:///C:/Users/PC/AppData/Local/Temp/BNZ.682fbebadada5/3.%20[Design]%20Example%203.pdf
https://brownbears.tistory.com/577
https://brownbears.tistory.com/511
https://sabarada.tistory.com/192